

PyBullet Reference

Ahmed Anwar

1 Simulation Connection

- **p.connect(mode)**
 - Connects to PyBullet physics server
 - `mode=p.GUI`: GUI mode
 - `mode=p.DIRECT`: Headless mode
- **p.disconnect()**
 - Ends connection to server
- **p.resetSimulation()**
 - Clears all objects and resets simulation
- **p.setRealTimeSimulation(flag)**
 - Enables real-time stepping (1) or manual stepping (0)
- **p.setPhysicsEngineParameter(...)**
 - Adjusts solver iterations, contact thresholds, etc.

```
p.connect(p.GUI)
p.setRealTimeSimulation(0)
p.setPhysicsEngineParameter(numSolverIterations=50)
```

2 Physics Parameters

- **p.setGravity(x, y, z)**: Sets gravity vector (m/s^2)
- **p.setTimeStep(dt)**: Sets simulation time step in seconds
- **p.setAdditionalSearchPath(path)**: Adds folder for models (URDF/SDF/MJCF)

```
p.setGravity(0,0,-9.8)
p.setTimeStep(1/240)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
```

3 Model Loading

- **p.loadURDF(fileName, basePosition, useFixedBase)**
 - Loads URDF object/robot
 - `useFixedBase=True` prevents falling
- **p.loadSDF(fileName)**: Load SDF model(s)

- **p.loadMJCF(fileName)**: Load MJCF robot model

```
plane = p.loadURDF("plane.urdf")
robot = p.loadURDF("kuka_iiwa/model.urdf", useFixedBase=True)
cube = p.loadURDF("cube_small.urdf", [0,0,0.5])
```

4 Simulation Loop

- **p.stepSimulation()**: Advances simulation by one time step
- **time.sleep(dt)** optional for visualization in GUI

```
for _ in range(1000):
    p.stepSimulation()
    time.sleep(1/240)
```

5 Joint Control

- **p.setJointMotorControl2(...)**
 - Controls joint behavior
 - Modes: `POSITION_CONTROL`, `VELOCITY_CONTROL`, `TORQUE_CONTROL`
- **p.getJointState(body, jointIndex)**
 - Returns (position, velocity, reactionForce, appliedTorque)
- **p.getNumJoints(body)**: Number of joints
- **p.getJointInfo(body, jointIndex)**: Returns joint metadata

```
p.setJointMotorControl2(robot, 0, p.POSITION_CONTROL,
    targetPosition=1.0, force=10)
pos, vel, reactionForce, torque = p.getJointState(robot, 0)
num_joints = p.getNumJoints(robot)
info = p.getJointInfo(robot, 0)
```

6 Base / Body Control

- **p.resetBasePositionAndOrientation(body, pos, orn)**: Teleports object
- **p.getBasePositionAndOrientation(body)**: Gets base position and orientation

- **p.resetBaseVelocity(body, linearVelocity, angularVelocity):** Sets linear and angular velocities
- **p.getBaseVelocity(body):** Returns current velocities

```
view = p.computeViewMatrix(cameraEyePosition=[1,1,1],
                           cameraTargetPosition=[0,0,0],
                           cameraUpVector=[0,0,1])
proj = p.computeProjectionMatrixFOV(fov=60, aspect=1.0,
                                    nearVal=0.1, farVal=3.1)
img = p.getCameraImage(320,240,view,proj)
rgb = img[2]; depth = img[3]; seg = img[4]
```

7 External Forces and Torques

- **p.applyExternalForce(body, linkIndex, forceObj, posObj, flags):** Apply continuous force
- **p.applyExternalTorque(body, linkIndex, torqueObj, flags):** Apply torque
- **p.applyImpulse(body, linkIndex, impulseObj, posObj):** Apply instantaneous impulse

```
p.applyExternalForce(cube, -1, [10,0,0], [0,0,0], p.WORLD_FRAME)
```

8 Constraints and Joints

- **p.createConstraint(...):** Create a constraint between two bodies
- **p.changeConstraint(cid, maxForce):** Modify constraint parameters
- **p.removeConstraint(cid):** Remove constraint

```
cid = p.createConstraint(robot, 0, cube, -1,
                        jointType=p.JOINT_FIXED, jointAxis=[0,0,0],
                        parentFramePosition=[0,0,0],
                        childFramePosition=[0,0,0])
p.changeConstraint(cid, maxForce=50)
```

9 Collision and Contact Info

- **p.getContactPoints(bodyA, bodyB):** Returns contact details
- **p.getClosestPoints(bodyA, bodyB, distance):** Returns points within distance
- **p.getAABB(body):** Returns axis-aligned bounding box

10 Ray Casting and Sensors

- **p.rayTest(rayFrom, rayTo):** Single raycast
- **p.rayTestBatch(rayFromList, rayToList):** Multiple rays
- **Camera Simulation:** Use view and projection matrices for RGB/depth/seg images

11 Debug and Visualization

- **p.configureDebugVisualizer(flag, value):** Enable/disable GUI buffers
- **p.addUserDebugLine(start, end, color, width):** Draw line
- **p.addUserDebugText(text, position, textColorOrRGB):** Draw text
- **p.changeVisualShape(body, linkIndex, rgbaColor):** Change color of object

12 Random Objects and Dynamics

- Generate multiple objects at random positions
- Apply random forces for physics experiments

```
import random
for i in range(5):
    x, y, z = random.uniform(-1,1), random.uniform(-1,1), 1
    p.loadURDF("cube_small.urdf", [x,y,z])
for body in bodies:
    fx, fy, fz = random.uniform(-10,10), 0, 0
    p.applyExternalForce(body, -1, [fx, fy, fz], [0,0,0], p.WORLD_FRAME)
```